



Customer Churn Prediction

Dataset summary

```
RangeIndex: 3749 entries, 0 to 3748
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0            3749 non-null   int64
1   CustomerID            3749 non-null   object
2   Age                   3562 non-null   float64
3   Gender                3749 non-null   object
4   Tenure                3749 non-null   int64
5   Service_Internet      3028 non-null   object
6   Service_Phone         3749 non-null   object
7   Service_TV            3749 non-null   object
8   Contract              3749 non-null   object
9   PaymentMethod         3562 non-null   object
10  MonthlyCharges         3749 non-null   float64
11  TotalCharges           3749 non-null   float64
12  StreamingMovies        3749 non-null   object
13  StreamingMusic         3749 non-null   object
14  OnlineSecurity         3749 non-null   object
15  TechSupport           3749 non-null   object
16  Churn                  3749 non-null   object
dtypes: float64(3), int64(2), object(12)
```

	Unnamed: 0	Age	Tenure	MonthlyCharges	TotalCharges
count	3749.000000	3562.000000	3749.000000	3749.000000	3749.000000
mean	1874.000000	43.655531	36.264070	75.844318	2718.968266
std	1082.387408	14.914474	20.505528	73.062971	3211.879149
min	0.000000	18.000000	1.000000	20.000000	13.190000
25%	937.000000	31.000000	19.000000	44.570000	1076.240000
50%	1874.000000	44.000000	36.000000	69.590000	2132.260000
75%	2811.000000	56.000000	54.000000	95.540000	3619.710000
max	3748.000000	69.000000	71.000000	1179.300000	79951.800000

Basic statistics of the dataset:

- Customers are aged between 18 and 69, with an average of 44.
- Tenure ranges from 1 to 71 months, averaging 36 months.
- Monthly Charges and Total Charges show high variance, indicating possible outliers.

```
Unnamed: 0      0
CustomerID      0
Age            187
Gender          0
Tenure          0
Service_Internet 721
Service_Phone   0
Service_TV      0
Contract        0
PaymentMethod   187
MonthlyCharges  0
TotalCharges    0
StreamingMovies  0
StreamingMusic   0
OnlineSecurity  0
TechSupport     0
Churn           0
dtype: int64
```

Handling missing values

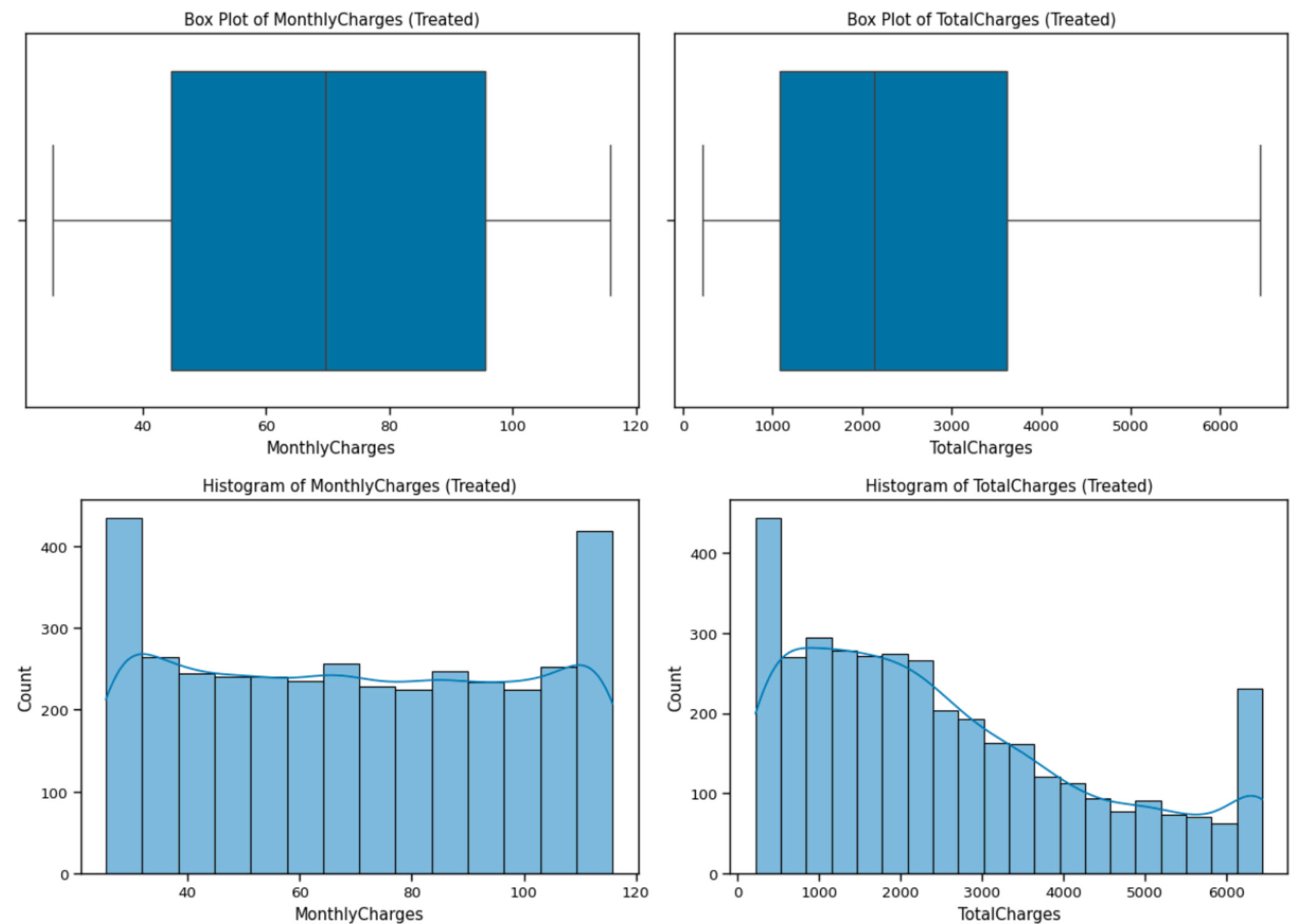
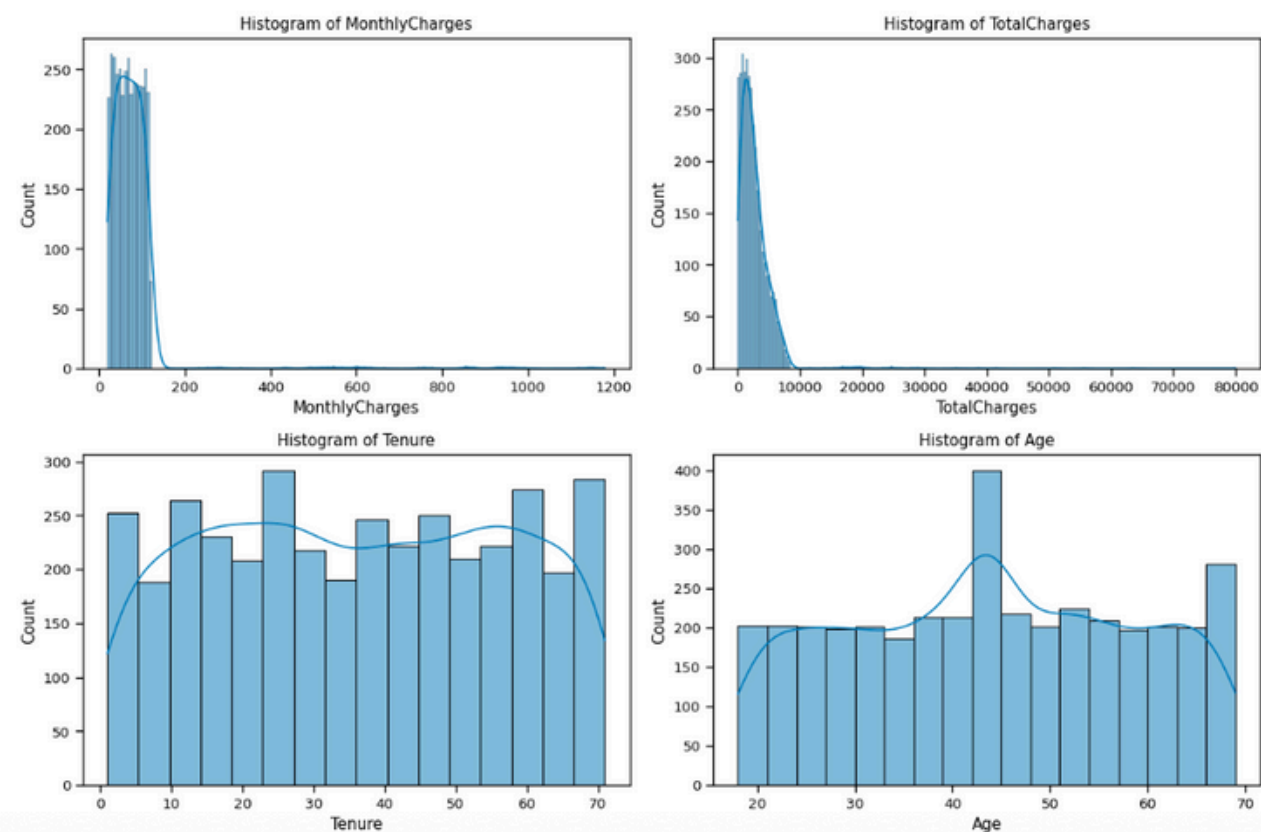
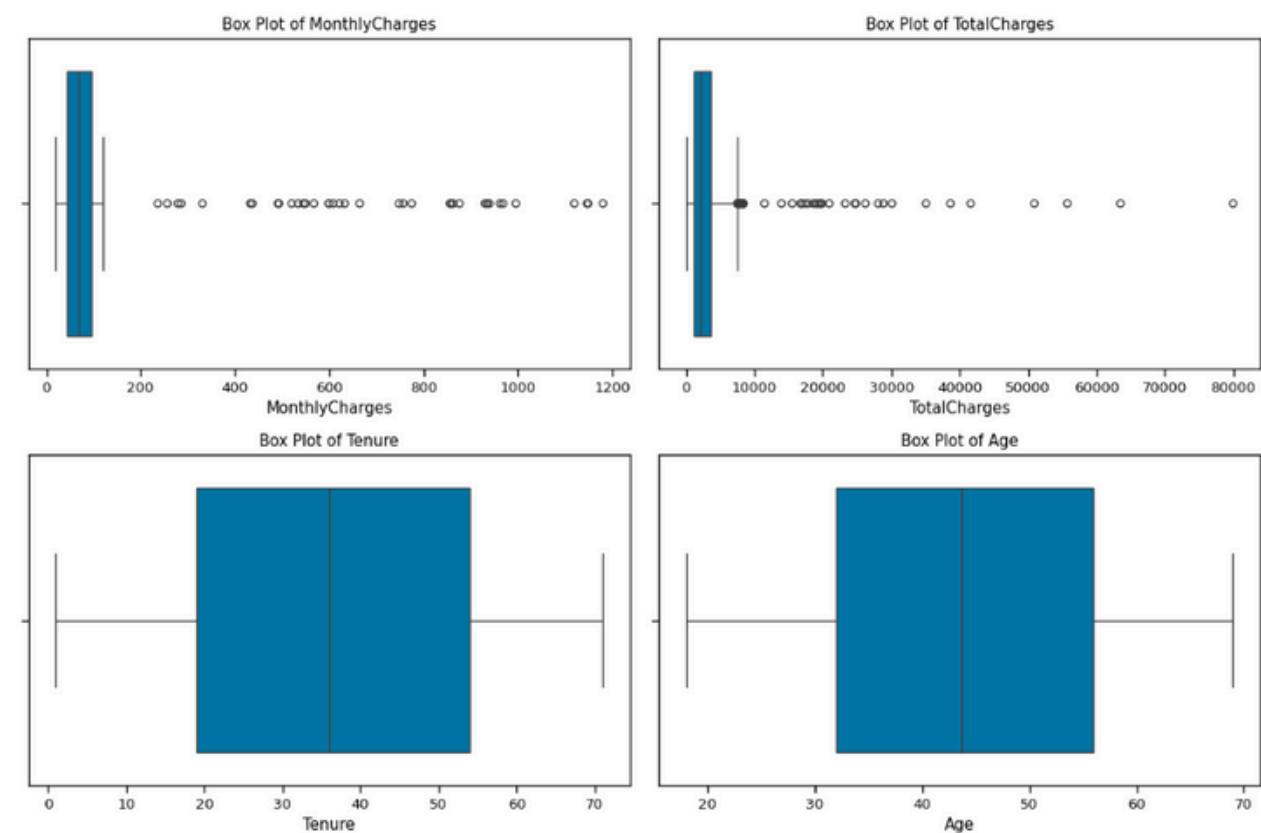
```
age_imputer = SimpleImputer(strategy='mean')
df['Age'] = age_imputer.fit_transform(df[['Age']])
```

```
# Service_Internet: Replace missing values with a new category
df['Service_Internet'].fillna('Unknown', inplace=True)
```

```
# PaymentMethod: Impute missing values using mode imputation
payment_imputer = SimpleImputer(strategy='most_frequent')
```

```
Missing Values After Handling:
CustomerID      0
Age             0
Gender          0
Tenure          0
Service_Internet 0
Service_Phone   0
Service_TV      0
Contract        0
PaymentMethod   0
MonthlyCharges  0
TotalCharges    0
StreamingMovies  0
StreamingMusic   0
OnlineSecurity  0
TechSupport     0
Churn           0
dtype: int64
```

Outlier Detection & Treatment



Encode categorical variables

```
# Define the columns to map
columns_to_map = ['OnlineSecurity', 'TechSupport', 'Churn']

# Define the mapping dictionary
mapping_dict = {'No': False, 'Yes': True}

# Map 'No' and 'Yes' to 0 and 1 for the specified columns
df[columns_to_map] = df[columns_to_map].replace(mapping_dict)

# Display the updated DataFrame
df.head()
```

I encoded the binary categorical columns OnlineSecurity, TechSupport, and Churn by mapping 'Yes' to True and 'No' to False.

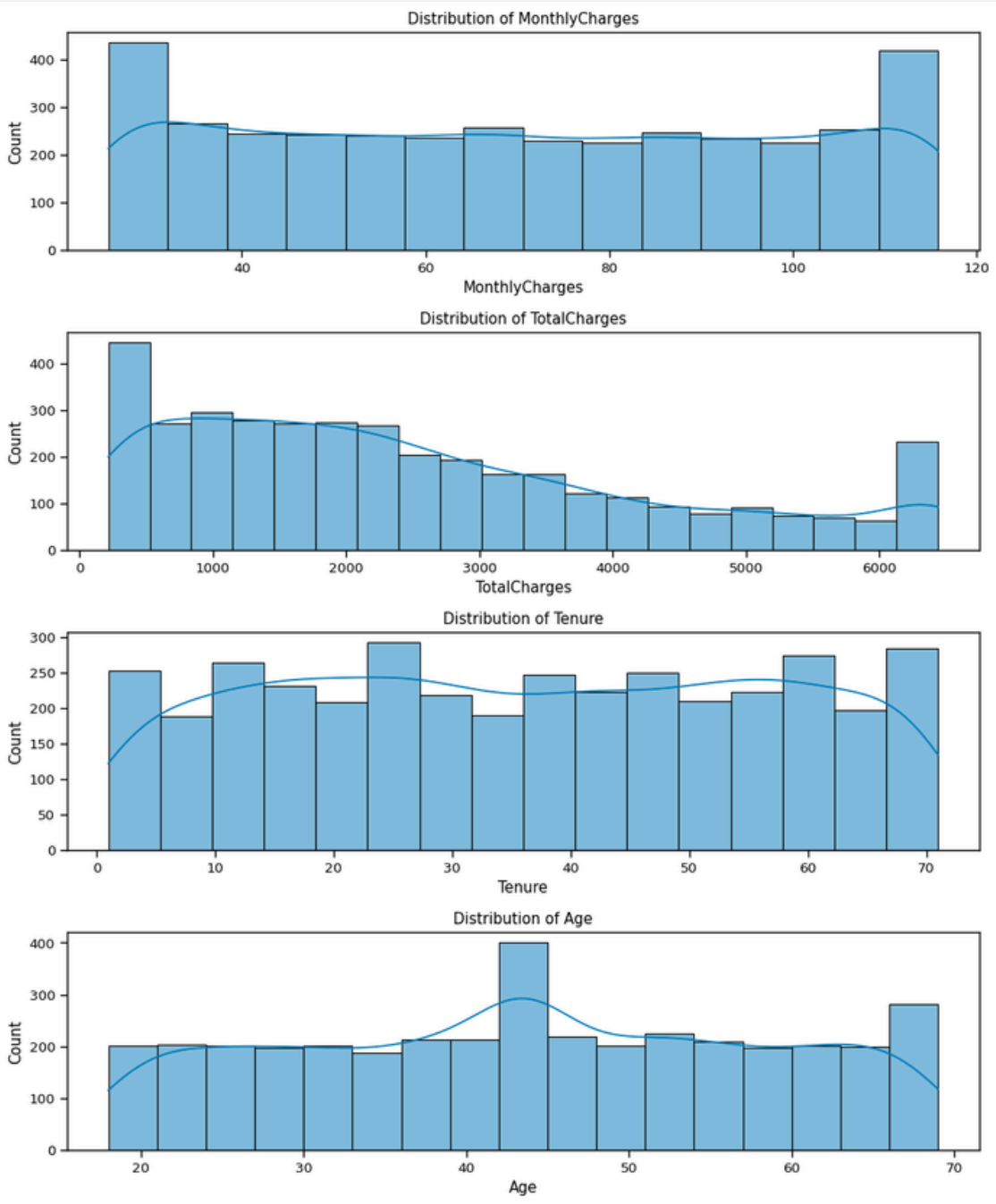


I applied one-hot encoding to the categorical columns Contract and PaymentMethod. I did this to convert each unique category within these columns into its own binary (0 or 1) feature

```
# Perform one-hot encoding for categorical variables
columns_to_encode = ['Contract', 'PaymentMethod']
df = pd.get_dummies(df, columns=columns_to_encode)

df.head()
```

Min-Max Scaling



```
from sklearn.preprocessing import MinMaxScaler

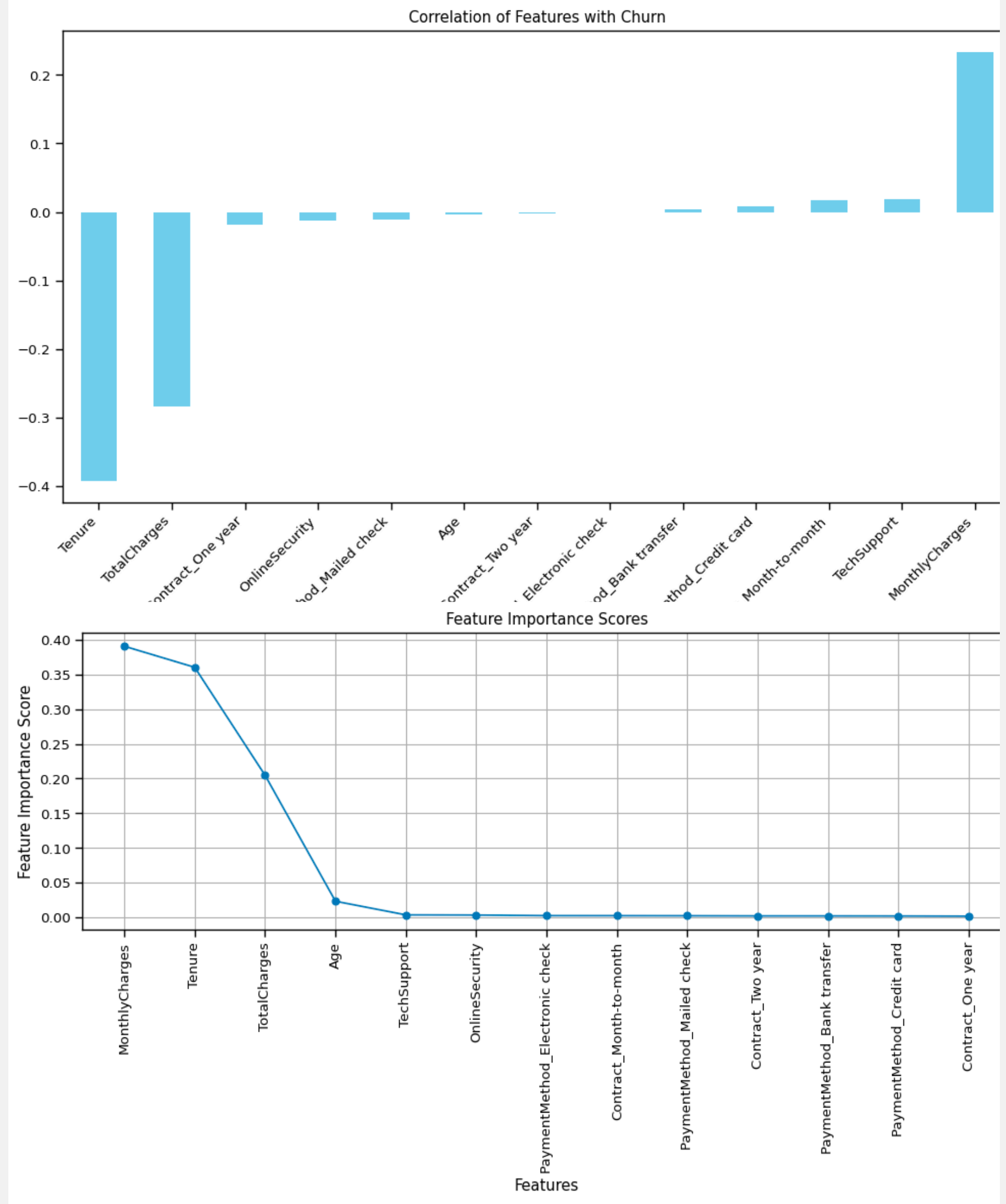
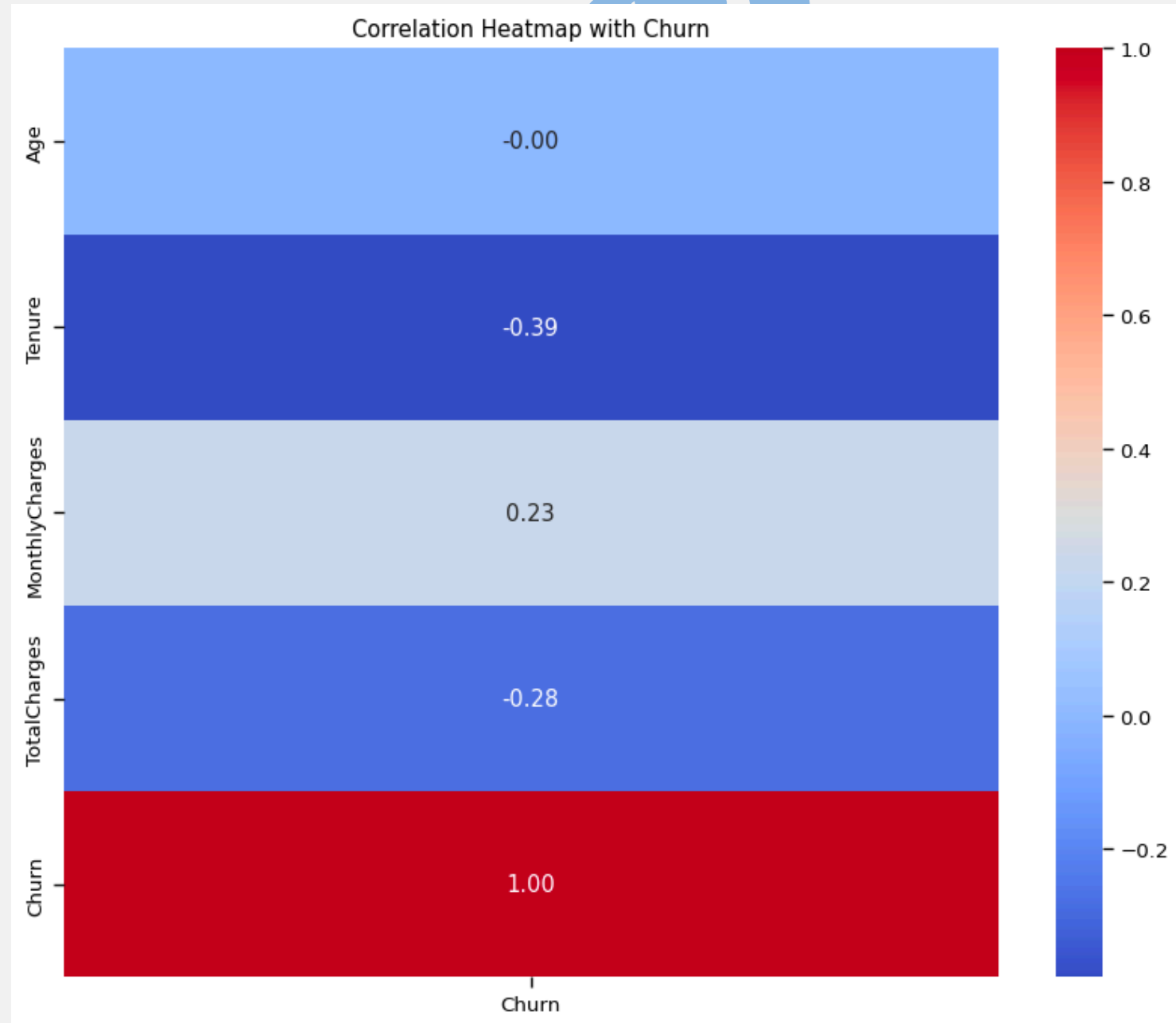
# Min-Max Scaling
scaler = MinMaxScaler()
df[numerical_features] = scaler.fit_transform(df[numerical_features])

# Display normalized or scaled dataset
df.head()
```

I applied Min-Max Scaling to the selected numerical features to bring them to a common scale between 0 and 1. By applying this technique to variables such as Age, Tenure, MonthlyCharges, and TotalCharges, I ensured that all features contributed equally during the training phase and avoided any imbalance due to differing value ranges.

CustomerID	Age	Gender	Tenure	Service_Inter	Service_Phor	Service_TV	MonthlyChar	TotalCharges	StreamingMc	StreamingMu	OnlineSecur	TechSupport	Churn	Contract_Mo	Contract_On	Contract_Tw	PaymentMet	PaymentMet	PaymentMet	PaymentMet
08729464-bc	0.745098	Male	0.1714286	DSL	Yes	No	0.5137976	0.1146984	No	No	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	TRUE
af95bc95-ba	1	Male	0.1714286	DSL	No	Yes	0.9465357	0.1977523	Yes	Yes	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	TRUE
1fe7eee6-22	0.5490196	Male	0.8428571	Fiber optic	No	Yes	1	1	Yes	Yes	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE
f736fe7b-1b	0.2745098	Female	0.8	Fiber optic	Yes	Yes	0.5832835	0.6803072	No	Yes	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE

EDA



Feature engineering

CustomerID, Service_Internet, Gender, Service-specific features and one-hot encoded payment method columns were removed

```
# Convert 'yes'/'no' values to 1/0
df['Service_Phone'] = df['Service_Phone'].map({'Yes': 1, 'No': 0})
df['Service_TV'] = df['Service_TV'].map({'Yes': 1, 'No': 0})
df['StreamingMovies'] = df['StreamingMovies'].map({'Yes': 1, 'No': 0})
df['StreamingMusic'] = df['StreamingMusic'].map({'Yes': 1, 'No': 0})

# Create TotalServiceUsage by summing the binary columns
df['TotalServiceUsage'] = df['Service_Phone'] + df['Service_TV'] + df['StreamingMovies'] + d

# Display the updated DataFrame
df[['Service_Phone', 'Service_TV', 'StreamingMovies', 'StreamingMusic', 'TotalServiceUsage']]
```

Python

	Service_Phone	Service_TV	StreamingMovies	StreamingMusic	TotalServiceUsage
0	1	0	0	0	1
1	0	1	1	1	3
2	0	1	1	1	3
3	1	1	0	1	3
4	1	1	1	0	3

	Age	Tenure	MonthlyCharges	TotalCharges	OnlineSecurity	TechSupport	Churn	TotalServiceUsage	ContractLength
0	0.745098	0.171429	0.513798	0.114698	1	0	0	1	12
1	1.000000	0.171429	0.946536	0.197752	0	0	0	3	24
2	0.549020	0.842857	1.000000	1.000000	0	0	0	3	1
3	0.274510	0.800000	0.583284	0.680307	0	1	0	3	1
4	0.823529	0.728571	0.054062	0.217235	1	1	0	3	24

all boolean columns (i.e., those with True/False values) were converted to integers (1/0) using a loop

Another engineered feature introduced was ContractLength, representing the duration of a customer's contract in months

```
# Mapping for contract durations
contract_length = {
    'Month-to-month': 1,
    'One year': 12,
    'Two year': 24
}

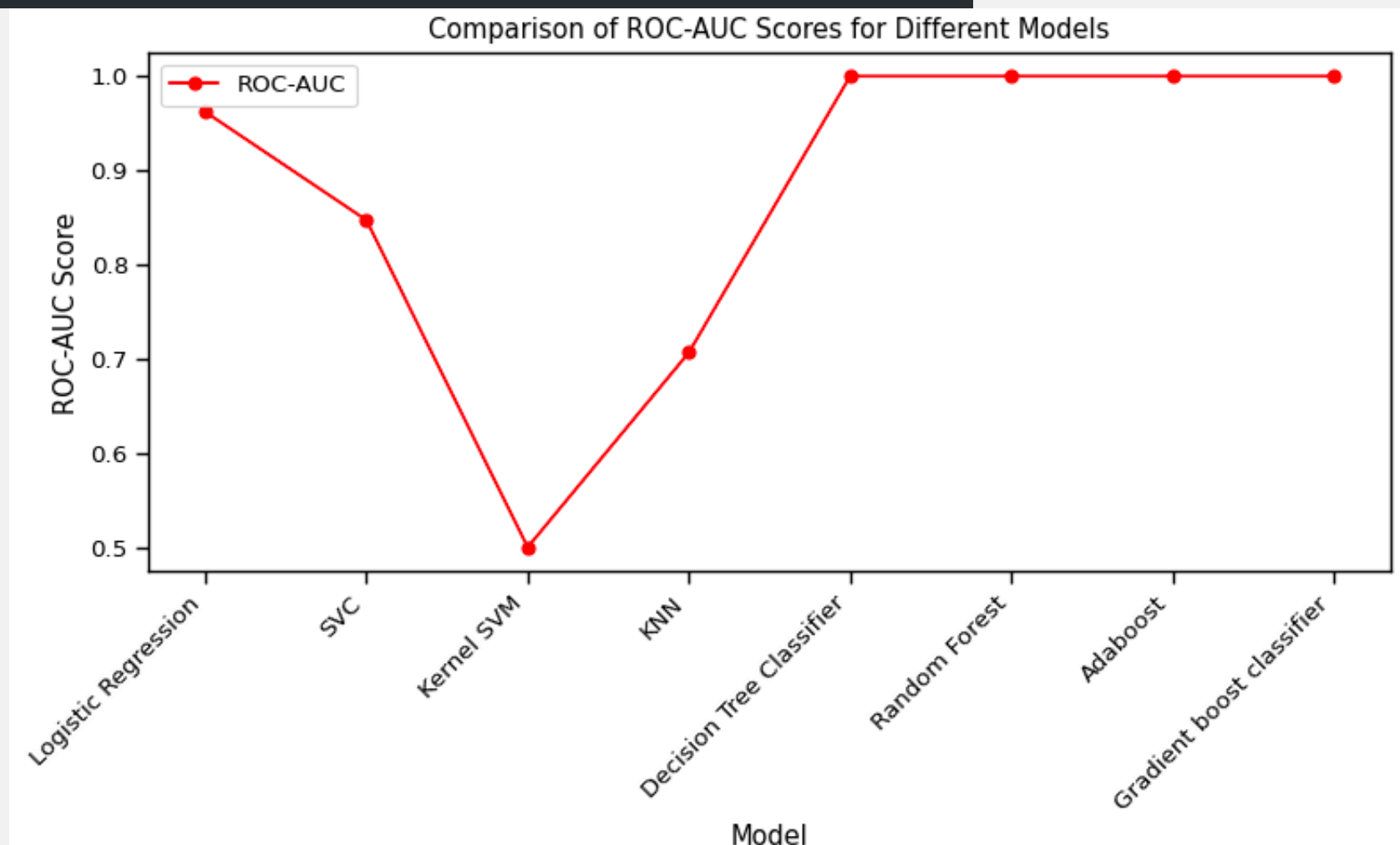
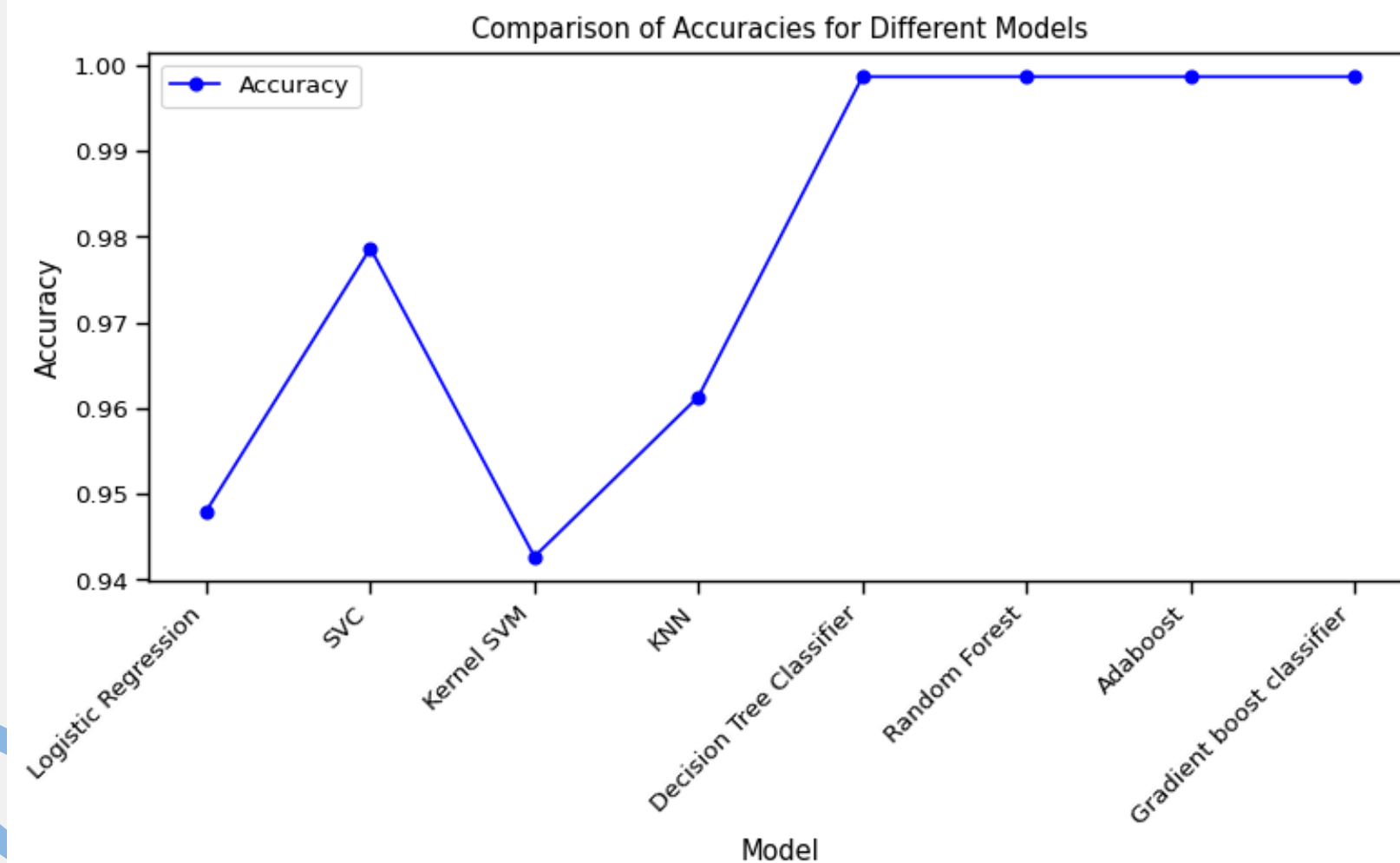
# Create the new column 'ContractLength' based on the existing contract columns
df['ContractLength'] = df[['Contract_Month-to-month', 'Contract_One year', 'Contract_Two year']].idxmax(axis=1)
df['ContractLength'] = df['ContractLength'].map({
    'Contract_Month-to-month': contract_length['Month-to-month'],
    'Contract_One year': contract_length['One year'],
    'Contract_Two year': contract_length['Two year']
})

# Drop the original contract columns
df.drop(columns=['Contract_Month-to-month', 'Contract_One year', 'Contract_Two year'], inplace=True)
```


Model evaluation

I first divided the cleaned dataset into training and testing sets using an 80/20 split. The training set was used to fit the models, while the testing set was reserved for evaluating their generalization performance. The target variable for prediction was Churn, and the remaining columns served as input features.

	Accuracy	Precision	Recall	F1-score	ROC-AUC	Cross-Validation Score
Logistic Regression	0.948000	0.525000	0.976744	0.682927	0.961498	0.928645
SVC	0.978667	0.909091	0.697674	0.789474	0.846716	0.977994
Kernel SVM	0.942667	0.000000	0.000000	0.000000	0.500000	0.930644
KNN	0.961333	0.818182	0.418605	0.553846	0.706473	0.949649
Decision Tree Classifier	0.998667	0.977273	1.000000	0.988506	0.999293	0.998999
Random Forest	0.998667	0.977273	1.000000	0.988506	0.999293	0.999333
Adaboost	0.998667	0.977273	1.000000	0.988506	0.999293	0.999666
Gradient boost classifier	0.998667	0.977273	1.000000	0.988506	0.999293	0.998999



Decision tree with GridSearch

```
from sklearn.model_selection import GridSearchCV

# Define the hyperparameter grid
param_grid = {
    'max_depth': [3, 5, 7],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'criterion': ['gini', 'entropy']
}

# Initialize the grid search with the classifier and hyperparameter grid
grid_search = GridSearchCV(estimator=DecisionTreeClassifier(), param_grid=param_grid, cv=5, scoring='accuracy')

# Perform grid search
grid_search.fit(X_train, y_train)

# Get the best hyperparameters
best_params = grid_search.best_params_

# Get the best model
best_model = grid_search.best_estimator_

# Evaluate the best model
best_model_score = best_model.score(X_test, y_test)

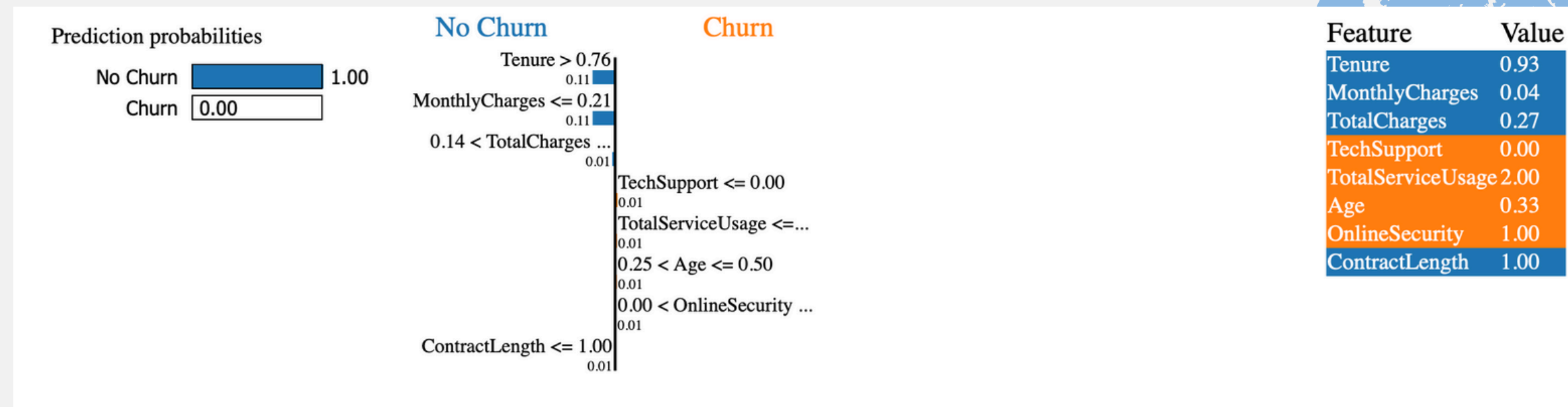
# Print the best hyperparameters and model score
print("Best Hyperparameters:", best_params)
print("Best Model Score:", best_model_score)
```

```
Best Hyperparameters: {'criterion': 'gini', 'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split': 5}
Best Model Score: 0.9986666666666667
```

Hyperparameter tuning was applied to the Decision Tree model using GridSearchCV, but the tuned model showed identical cross-validation accuracy to the default one ($\approx 99.9\%$), indicating that the default tree was already highly optimized for this dataset.

Using these optimal hyperparameters, the model achieved a test accuracy of 0.9987, indicating a near-perfect fit. This step confirms the importance of hyperparameter tuning, as it allows the model to generalize better while avoiding overfitting.

Interpretation with LIME



This LIME visualization shows the prediction explanation for a single customer. The model predicts with near certainty (100%) that the customer will not churn, primarily due to:

High Tenure (0.93) and low Monthly Charges (0.04), which strongly reduce churn likelihood.
Features like Contract Length and Total Charges also support retention.

Some churn-indicative factors (e.g., no Tech Support, low Age) are present, but their influence is outweighed.

LIME helps interpret individual predictions, making the model more transparent for business decision-making.

The background is a light gray color, decorated with various hand-drawn blue doodles. These include several loops and swirls at the top, a series of vertical lines on the right side, a wavy line at the bottom center, and several checkmarks at the bottom right. The text is centered in the middle of the image.

**Thank you
very much!**